

Low-Level Design

DOCUMENT :: 2026-08-06

Low-Level Design — CREDENCE (LMS Prototype)

Date: 2026-08-05 Companion: [HLD](#)

1. Module map

```
apps/api/src/
├─ server.ts           # Process entry: env, buildApp, migrate+seed, listen
├─ app.ts              # Fastify factory: plugins, decorators, routes, static
├─ db/
│  ├─ client.ts        # better-sqlite3 + Drizzle, reads DATABASE_URL
│  ├─ schema.ts         # Table definitions (§2)
│  ├─ migrate.ts        # Applies Drizzle migrations
│  └─ seed.ts           # Idempotent upsert: admin + 2 applicants + 1 mid-flight loan
├─ domain/
│  ├─ amortization.ts   # calculateEmi(), generateSchedule() – pure
│  └─ underwriting.ts   # evaluate() – pure, returns approve/reject
├─ lib/
│  ├─ auth.ts           # bcrypt + JWT + cookie helpers
│  ├─ errors.ts          # AppError hierarchy + Fastify errorHandler
│  └─ money.ts           # roundCents(), formatINR()
└─ routes/
   ├─ auth.ts           # signup, login, logout, me, refresh
   ├─ applications.ts   # POST /applications, GET /applications[:id]
   ├─ loans.ts          # GET /loans[:id], POST /loans/:id/installments/:n/pay
   └─ admin.ts          # queue + decision + disburse
```

```
packages/shared/src/index.ts
  Zod schemas + DTOs (SignupInput, LoginInput, ApplicationInput, DecisionInput,
  UserDTO, ApplicationDTO, LoanDTO, InstallmentDTO, role/status enums)
```

```
apps/web/src/
├─ main.tsx            # Mounts React + QueryClient + Router
├─ App.tsx             # Routes + Toaster + session-expired listener
├─ lib/
│  ├─ api.ts           # apiFetch(): fetch with credentials + 401→refresh
│  └─ format.ts        # formatINR(), formatDate()
├─ hooks/              # TanStack Query wrappers (useMe, useApplications, ...)
├─ stores/             # Zustand store for dashboard tab/search
├─ components/
│  ├─ BrandLogo.tsx    # 
│  ├─ AppHeader.tsx    # Top nav for authenticated pages
│  ├─ AuthGuard.tsx    # Redirects unauthenticated/wrong-role
│  ├─ RedirectIfLoggedIn # Redirects already-authenticated visitors
│  ├─ EmptyState.tsx   # Card-based empty list state
│  ├─ ProgressBar.tsx  # role=progressbar, ARIA values
│  ├─ SectionHeader.tsx # icon + h2 + count badge
│  ├─ StatCard.tsx     # label + value + footer + ProgressBar
│  ├─ Money.tsx        # tabular-nums INR formatter
│  ├─ StatusBadge.tsx  # Status → badge color map
│  └─ ui/              # shadcn primitives (Button, Card, Form, Input, ...)
└─ routes/
   ├─ landing.tsx      # Hero + EMI calculator + feature pillars
   └─ login.tsx, signup.tsx
```

- └─ apply.tsx # 3-step loan application wizard
- └─ dashboard.tsx # Overview + applications + active loans
- └─ loan-detail.tsx # Installments table + Pay EMI
- └─ admin/
 - └─ queue.tsx
 - └─ application-detail.tsx

2. Database schema (Drizzle, SQLite)

All money is integer cents. All timestamps/dates are text ISO 8601. All IDs are text (nanoid, 21 chars). Foreign keys cascade on user delete.

```
// apps/api/src/db/schema.ts
export const users = sqliteTable('users', {
  id: text('id').primaryKey(),
  email: text('email').notNull().unique(),
  passwordHash: text('password_hash').notNull(),
  fullName: text('full_name').notNull(),
  role: text('role', { enum: ['applicant', 'admin'] }).notNull(),
  monthlyIncome: integer('monthly_income').notNull(), // cents
  createdAt: text('created_at').notNull(),
});

export const loanApplications = sqliteTable('loan_applications', {
  id: text('id').primaryKey(),
  userId: text('user_id').notNull().references(() => users.id),
  amountCents: integer('amount_cents').notNull(),
  termMonths: integer('term_months').notNull(),
  annualRateBps: integer('annual_rate_bps').notNull(),
  purpose: text('purpose').notNull(),
  employment: text('employment', { enum: ['salaried', 'self_employed'] }).notNull(),
  status: text('status', { enum: ['pending', 'approved', 'rejected', 'disbursed'] }).notNull(),
  decisionReason: text('decision_reason'),
  decidedBy: text('decided_by').references(() => users.id),
  decidedAt: text('decided_at'),
  disbursedAt: text('disbursed_at'),
  createdAt: text('created_at').notNull(),
});

export const loans = sqliteTable('loans', {
  id: text('id').primaryKey(), // = loanApplications.id
  applicationId: text('application_id').notNull().unique().references(() => loanApplications.id),
  userId: text('user_id').notNull().references(() => users.id),
  principalCents: integer('principal_cents').notNull(),
  annualRateBps: integer('annual_rate_bps').notNull(),
  termMonths: integer('term_months').notNull(),
  startDate: text('start_date').notNull(),
  endDate: text('end_date').notNull(),
  status: text('status', { enum: ['active', 'closed'] }).notNull(),
  outstandingCents: integer('outstanding_cents').notNull(),
});

export const installments = sqliteTable('installments', {
  id: text('id').primaryKey(),
  loanId: text('loan_id').notNull().references(() => loans.id),
  sequence: integer('sequence').notNull(),
  dueDate: text('due_date').notNull(),
});
```

```

principalDue: integer('principal_due').notNull(),
interestDue: integer('interest_due').notNull(),
paidAmount: integer('paid_amount').notNull().default(0),
paidAt:      text('paid_at'),
}, (t) => ({
  uniqLoanSeq: uniqueIndex('uniq_loan_seq').on(t.loanId, t.sequence),
}));

```

3. API contracts

All routes return JSON. Errors: { error: string, message?: string, issues?: ZodIssue[] }. Auth via cookie `lms_session` (httpOnly, SameSite=Lax). Status codes: 200 ok, 204 no-content, 400 validation, 401 no/invalid session, 403 wrong role, 404 not found / not owner, 409 domain conflict.

Method	Path	Auth	Purpose
POST	/api/auth/signup	none	Create applicant account, set cookie
POST	/api/auth/login	none	Verify password, set cookie
POST	/api/auth/logout	any	Clear both cookies
POST	/api/auth/refresh	refresh cookie	Issue new access + refresh tokens
GET	/api/auth/me	session	Current user
POST	/api/applications	any	Create application; auto-rejects if rule fails
GET	/api/applications	any	List current user's applications
GET	/api/applications/:id	owner / admin	Single application + rule recommendation
GET	/api/loans	any	Applicants see own; admins see all
GET	/api/loans/:id	owner / admin	Loan + installments
POST	/api/loans/:id/installments/:n/pay	owner	Mark installment paid, decrement outstanding
GET	/api/admin/applications?status=pending	admin	Review queue
POST	/api/admin/applications/:id/decision	admin	Approve / reject
POST	/api/admin/applications/:id/disburse	admin	Create loan + schedule

Zod validates every request body via schemas exported from `packages/shared`. A `ZodError` becomes 400 { error: 'ValidationError', issues }.

4. Amortization

`apps/api/src/domain/amortization.ts` — two pure functions.

```

export function calculateEmi(
  principalCents: number,
  annualRateBps: number,
  termMonths: number
): number; // integer cents, Math.round half-up

```

```

export interface ScheduleRow {
  sequence: number;          // 1..N
  dueDate: string;           // ISO yyyy-mm-dd
  principalDue: number;      // integer cents
  interestDue: number;       // integer cents
}

export function generateSchedule(
  principalCents: number,
  annualRateBps: number,
  termMonths: number,
  startDate: string
): ScheduleRow[]; // length = termMonths, sum(principalDue) === principalCents

```

Algorithm

1. $r = \text{annualRateBps} / 10000 / 12$ (monthly rate).
2. $\text{emi} = \text{roundCents}(P \cdot r \cdot (1+r)^N / ((1+r)^N - 1))$. Zero-rate edge: $\text{emi} = \text{roundCents}(P / N)$.
3. For $i = 1..N-1$: $\text{interest} = \text{roundCents}(\text{outstanding} \cdot r)$, $\text{principal} = \text{emi} - \text{interest}$, $\text{outstanding} -= \text{principal}$.
4. Final $i = N$: $\text{interest} = \text{roundCents}(\text{outstanding} \cdot r)$, $\text{principal} = \text{outstanding}$. The last row absorbs the rounding so the sum is exact.
5. $\text{dueDate} = \text{addMonths}(\text{startDate}, i)$ via `date-fns` with end-of-month clamp.

Worked example: ₹50,000, 15% APR, 12 months, start 2026-01-01 → $\text{emi} \approx ₹4,512.88$, schedule sums to ₹50,000.00 exactly.

5. Underwriting

`apps/api/src/domain/underwriting.ts:`

```

export function evaluate(
  monthlyIncomeCents: number,
  principalCents: number,
  annualRateBps: number,
  termMonths: number
): { recommendation: 'approve' | 'reject'; reason?: string };

```

Rules (first match wins):

1. $\text{monthlyIncomeCents} \leq 0 \rightarrow \text{reject "Income not provided"}$.
2. Compute $\text{emi} = \text{calculateEmi}(\dots)$.
3. $\text{emi} / \text{monthlyIncomeCents} > 0.50 \rightarrow \text{reject "EMI exceeds 50\% of income (FOIR)"}$.
4. $\text{monthlyIncomeCents} < 3 \cdot \text{emi} \rightarrow \text{reject "Income insufficient for requested EMI"}$.
5. Else approve.

The application is **always** persisted — either as pending (rule approved) or rejected (rule failed) with `decided_by = null`. The admin queue therefore only shows genuinely human-pending work.

6. State machine

From → To	Trigger	Side effects
(none) → pending	POST /applications, rule approved	INSERT row
(none) → rejected	POST /applications, rule rejected	INSERT + set decision_reason, decided_at
pending → approved	admin decision: 'approve'	set decided_by, decided_at
pending → rejected	admin decision: 'reject'	set decided_by, decided_at, decision_reason
approved → disbursed	admin disburse	INSERT loans + bulk INSERT installments, set disbursed_at

rejected and disbursed are terminal at the application level. Loan lifecycle continues on the `loans` row (active → closed when the last installment is paid).

7. Auth flow

- **Signup / login:** bcrypt-hash password → insert user (or verify) → sign access JWT (`lms_session`, 1h) and refresh JWT (`lms_refresh`, 30d) → set both as `httpOnly`, `SameSite=Lax`, `Secure` in production.
- **Authenticated request:** server reads `lms_session` cookie, verifies JWT, attaches `session = { id, role }` to the request.
- **Access expired:** frontend `apiFetch` catches 401, calls `POST /api/auth/refresh`. Server validates refresh cookie, issues fresh tokens. One in-flight refresh shared across concurrent calls to avoid stampede. If refresh fails, dispatch `lms:session-expired` and redirect to `/login`.
- **Logout:** clear both cookies.

8. Error model

```
// apps/api/src/lib/errors.ts
class AppError extends Error { status: number; code: string; }
class ValidationError extends AppError { status = 400; code = 'ValidationError'; }
class UnauthorizedError extends AppError { status = 401; code = 'Unauthorized'; }
class ForbiddenError extends AppError { status = 403; code = 'Forbidden'; }
class NotFoundError extends AppError { status = 404; code = 'NotFound'; }
class ConflictError extends AppError { status = 409; code: string; }

function errorHandler(err, req, reply) {
  if (err instanceof ZodError) return reply.code(400).send({ error: 'ValidationError', issues: err.issues });
  if (err instanceof AppError) return reply.code(err.status).send({ error: err.code, message: err.message });
  req.log.error(err);
  return reply.code(500).send({ error: 'InternalServerError' });
}
```

Specific conflict codes: `EmailAlreadyUsed`, `InvalidStateTransition`, `NoInstallmentFound`, `LoanClosed`.

9. Testing

Run with `pnpm --filter @lms/api test (node --test --import tsx test/*.test.ts)`. No test framework — `node:test` with in-memory SQLite.

File	Coverage
<code>db.test.ts</code>	Migrations create tables; reset clears cache
<code>auth.test.ts</code>	Signup sets cookie, rejects duplicate email; login round-trip; me unauthenticated 401
<code>underwriting.test.ts</code>	Rejects zero income, FOIR > 50%, income < 3× EMI; approves healthy apps and boundary cases
<code>amortization.test.ts</code>	Sum equals principal; last row absorbs rounding; zero-rate edge
<code>applications.test.ts</code>	Owner-only access; admin override; rule recommendation surfaced
<code>loans.test.ts</code>	Owner sees schedule; pay first installment decrements outstanding; pay again 409; pay last closes loan
<code>admin.test.ts</code>	Admin queue; decision transitions; disburse creates loan + schedule
<code>health.test.ts</code>	<code>/api/health</code> returns <code>{ ok: true }</code>

10. Build & deploy

Command	Effect
<code>pnpm install --frozen-lockfile</code>	Install all workspaces
<code>pnpm -r build</code>	Build web (<code>tsc --noEmit && vite build</code>) + API (<code>tsc</code>)
<code>pnpm --filter @lms/api start:prod</code>	Run prod server locally (uses built dist, <code>NODE_ENV=production</code>)
<code>pnpm dev</code>	Run web (Vite) + API (<code>tsx watch</code>) in parallel
<code>pnpm --filter @lms/api test</code>	Run all API tests
<code>pnpm pdf</code>	Regenerate <code>docs/hld/hld.pdf</code> and <code>docs/lld/lld.pdf</code> from the markdown

Render: single Node service, free tier, persistent disk at `/data`, `DATABASE_URL=/data/lms.db`. Push to `main` auto-deploys. On boot, the server runs `migrate` then `seed` (idempotent upsert), so a cold start always ends with a usable DB.

End of LLD. See the [HLD](#) for the system overview and deployment topology.